Unit: Servlet Annotations Configuration — `@WebServlet`

Video Title: *#6 Servlet Annotations Configurations || @WebServlet Annotation || No need of web.xml*
Video Link: https://youtu.be/XuXio4NXoWE?si=mjj-STlR9V8c2Nxw

This topic explains how **Servlets can be configured using annotations** instead of a `web.xml` deployment descriptor. The main annotation discussed is **`@WebServlet`**.

1. Introduction

In traditional Java web applications, Servlet URL mapping was commonly done using the **`web.xml`** file.

For example, to connect:

```
/aaa → MyServlet
```

we had to write Servlet configuration and mapping inside `web.xml`.

Modern Servlet technology provides **annotations**, which allow configuration to be written directly inside the Java code.

The **`@WebServlet`** annotation can be used to map a URL to a Servlet without manually creating a `web.xml` file.

Definition

Annotation: An annotation is metadata written in Java source code that provides additional information to the compiler, server, or framework.

`@WebServlet`: A Servlet annotation used to configure a Servlet and specify the URL pattern that should invoke it.

2. Servlet Configuration Using `web.xml`

In the traditional approach, the Servlet and its URL mapping are configured in `web.xml`.

For example:

```
/aaa

 ↓
```

`MyServlet`

The server reads the deployment descriptor and determines which Servlet should handle the request.

This approach requires separate XML configuration.

Key Points

- `web.xml` is known as the **Deployment Descriptor**.
- Servlet mappings can be defined in `web.xml`.
- The configuration is separate from the Java Servlet code.
- Older Java web projects commonly use this approach.

3. Servlet Configuration Using Annotations

With annotations, the Servlet mapping can be written directly above the Servlet class.

Example:

```
@WebServlet("/aaa")

public class MyServlet extends HttpServlet {



}
```

Here:

`/aaa`

is the URL pattern associated with `MyServlet`.

The server can identify this mapping from the annotation.

Important for Exam

`@WebServlet` can be used to configure and map a Servlet without manually defining the mapping in `web.xml`.

4. What is `@WebServlet`?

Definition

`@WebServlet`: An annotation provided by the Servlet API that is used to declare a Java class as a Servlet and specify its URL mapping.

The basic syntax is:

```
@WebServlet("/url-pattern")
```

Example:

```
@WebServlet("/aaa")
public class MyServlet extends HttpServlet {


}
```

This tells the server that requests matching `/aaa` should be handled by `MyServlet`.

5. Why Use `@WebServlet`?

The main advantage is that the Servlet configuration can be kept **directly with the Java Servlet class**.

Instead of maintaining a separate XML mapping, we can write:

```
@WebServlet("/aaa")
```

above the class.

Advantages

- **Simple:** Less configuration code is required.
- **Cleaner:** Configuration stays close to the Servlet code.
- **Faster:** URL mapping can be added directly to the class.
- **Modern:** Annotations are widely used in modern Servlet applications.
- **No separate mapping required:** For this basic configuration, the URL mapping does not need to be written manually in `web.xml`.

6. Step-by-Step Example

Step 1: Create a Dynamic Web Project

In Eclipse, create a **Dynamic Web Project**.

During project creation, there may be an option:

Generate web.xml deployment descriptor

For an annotation-based configuration, this option can be **unchecked**.

This allows the project to be created without generating the traditional `web.xml` deployment descriptor.

Step 2: Create `index.html`

Create an HTML page named:

`index.html`

Add a link:

```
<a href="aaa">Click Me</a>
```

Explanation

When the user clicks **Click Me**, the browser sends a request for the URL associated with:

`aaa`

The Servlet must be mapped to this URL for the request to work.

7. Create the Servlet Class

Create a Java class called:

`MyServlet`

The class should extend:

`HttpServlet`

Example:

```
public class MyServlet extends HttpServlet {


}
```

`HttpServlet` provides functionality for creating Servlets that handle HTTP requests.

8. Override the `service()` Method

The Servlet can override the `service()` method to handle the incoming request.

Example:

```
protected void service(HttpServletRequest request,
                       HttpServletResponse response) {


    System.out.println("I am in MyServlet service method");
}
```

The `service()` method receives:

- `HttpServletRequest` → information about the client's request.
- `HttpServletResponse` → used to provide a response to the client.

For the demonstration in the video, a simple `System.out.println()` statement is used to show that the Servlet's `service()` method is executing.

9. What Happens Without `@WebServlet`?

Suppose we have:

```
<a href="aaa">Click Me</a>
```

and:

```
public class MyServlet extends HttpServlet {


    protected void service(HttpServletRequest request,
                           HttpServletResponse response) {
        System.out.println("I am in MyServlet service method");
    }
}
```

If there is no `web.xml` mapping and no Servlet annotation, the server does not know that:

```
/aaa
```

should be handled by:

```
MyServlet
```

Therefore, when the link is clicked, the application can produce a:

404 Not Found error.

Why?

Because the requested URL has not been mapped to the Servlet.

10. Add the `@WebServlet` Annotation

To solve the mapping problem, add:

```
@WebServlet("/aaa")
```

above the Servlet class.

Example:

```
@WebServlet("/aaa")
public class MyServlet extends HttpServlet {


    protected void service(HttpServletRequest request,
                           HttpServletResponse response) {


        System.out.println("I am in MyServlet service method");
    }
}
```

Now the mapping is:

```
/aaa
  ↓
MyServlet
```

The server can identify the Servlet associated with the URL.

11. Complete Example

A simplified Servlet example is:

```
import javax.servlet.annotation.WebServlet;
```

```
import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;


@WebServlet("/aaa")

public class MyServlet extends HttpServlet {


    protected void service(HttpServletRequest request,

                           HttpServletResponse response) {


        System.out.println("I am in MyServlet service method");

    }

}
```

The HTML page contains:

```
<a href="aaa">Click Me</a>
```

Working

```
User clicks "Click Me"

          ↓

Request for /aaa

          ↓

@WebServlet("/aaa")

          ↓

MyServlet

          ↓

service() method executes
```

The console can then display:

```
I am in MyServlet service method
```

12. Importance of the `/` in `@WebServlet`

The URL pattern should be written with a **forward slash**.

Correct:

```
@WebServlet("/aaa")
```

The slash is an important part of the URL pattern.

Remember

```
/aaa

 ↑

Forward slash
```

For examination purposes, remember the basic syntax:

```
@WebServlet("/url-pattern")
```

13. `web.xml` vs `@WebServlet`

Both approaches can be used to configure Servlet URL mappings.

Feature	`web.xml`	`@WebServlet`
Configuration type	XML	Java annotation
Location	`WEB-INF/web.xml`	Above Servlet class
URL mapping	`<url-pattern>`	Annotation value
Servlet class mapping	`<servlet-class>`	Java class itself
Configuration style	Separate XML file	Directly in Java code
Modern usage	Common in older/traditional projects	Common in modern Servlet applications

Example

Using `web.xml`:

```
<servlet-mapping>

    <servlet-name>myservlet</servlet-name>

    <url-pattern>/aaa</url-pattern>

</servlet-mapping>
```

Using annotation:

```
@WebServlet("/aaa")

public class MyServlet extends HttpServlet {

}
```

The annotation approach is much shorter for a simple Servlet mapping.

14. How the Complete Process Works

Consider the following HTML:

```
<a href="aaa">Click Me</a>
```

and Servlet:

```
@WebServlet("/aaa")

public class MyServlet extends HttpServlet {


    protected void service(HttpServletRequest request,

                           HttpServletResponse response) {

        System.out.println("I am in MyServlet service method");

    }

}
```

The complete process is:

1. The user opens `index.html`.
2. The user clicks **Click Me**.
3. The browser sends a request for `/aaa`.
4. The Servlet container checks the Servlet mappings.
5. It finds `@WebServlet("/aaa")`.
6. The mapping points to `MyServlet`.
7. `MyServlet` handles the request.
8. Its `service()` method executes.

15. Important Exam Points

Important for Exam:

- **`@WebServlet`** is used for Servlet configuration and URL mapping.
- It can eliminate the need to manually configure Servlet mappings in `web.xml`.
- The annotation is written **above the Servlet class**.
- The basic syntax is:

```
@WebServlet("/aaa")
```

- The URL pattern contains a **forward slash `/`**.
- The Servlet class generally extends **`HttpServlet`**.
- Without a proper mapping, requesting a Servlet URL can result in a **404 Not Found** error.
- Annotations keep configuration close to the Java code.
- `web.xml` is the traditional deployment descriptor approach.

16. Commonly Confused

`web.xml` and `@WebServlet`

Do not think that `@WebServlet` and `web.xml` are two different types of Servlets.

They are **two different ways of configuring/mapping a Servlet**.

```
Traditional approach

        ↓

    web.xml

        ↓

 Servlet Mapping




Annotation approach

        ↓

 @WebServlet("/aaa")

        ↓

 Servlet Mapping
```

Remember

`web.xml` → configuration in XML

`@WebServlet` → configuration in Java code

17. Key Points

- **`@WebServlet`** is a Servlet annotation.
- It is used to specify a **URL pattern for a Servlet**.
- It is placed directly above the Servlet class.
- Example:

```
@WebServlet("/aaa")

public class MyServlet extends HttpServlet {

}
```

- `/aaa` is the URL pattern.
- It provides a simpler alternative to manually defining basic Servlet mappings in `web.xml`.
- Without a valid mapping, the requested Servlet URL may result in **404 Not Found**.
- The Servlet can process the request using methods such as `service()`.
- Annotation-based configuration is cleaner and keeps configuration close to the Servlet code.

18. Video Information

Video Details

Video Number: #6

Video Title: *Servlet Annotations Configurations || @WebServlet Annotation || No need of web.xml*

Main Topic: Servlet configuration using annotations

Main Annotation: `@WebServlet`

Main Concept: Mapping a URL to a Servlet without manually creating/configuring `web.xml`.

Example URL Pattern: `/aaa`

Example Servlet: `MyServlet`

Video Link: https://youtu.be/XuXio4NXoWE?si=mjj-STlR9V8c2Nxw

Summary

`@WebServlet` provides a simple way to configure a Servlet directly in Java code. Instead of writing Servlet URL mappings separately in `web.xml`, we can write an annotation such as **`@WebServlet("/aaa")`** above the Servlet class. When a client requests `/aaa`, the Servlet container can identify `MyServlet` as the Servlet responsible for handling that request. This makes basic Servlet configuration shorter, cleaner, and easier to maintain.